CC3084 · Data Science · Sección 10 · Grupo 6

Gradio para dashboards interactivos

Investigación, comparación, arquitectura e implementación de un observatorio comercial reproducible

Universidad del Valle de Guatemala
Segundo semestre de 2026

Pablo Daniel Barillas Moreno Carné No. 222193

Carlos Aldana Carné No. 23394

Guatemala, octubre de 2026

Índice

1. Resumen ejecutivo **3**

2. Objetivos y alcance **3**

2.1. Objetivo general . . . 3

2.2. Objetivos específicos . . . 3

3. Dashboards interactivos **3**

4. Qué es Gradio **4**

4.1. Origen y propósito . . . 4

4.2. Lenguajes y ecosistema . . . 4

5. Modelo de programación **4**

5.1. Tres niveles principales . . . 5

5.2. Ejemplo mínimo . . . 5

6. Arquitectura y flujo de datos **5**

6.1. Estado . . . 6

6.2. Colas y concurrencia . . . 6

7. Comparación con otras herramientas **6**

8. Aplicación desarrollada por el grupo **7**

8.1. Variables y funcionalidades . . . 7

8.2. Cálculos principales . . . 8

8.3. Calidad y pruebas . . . 8

9. Diseño visual y accesibilidad **8**

10. Seguridad y privacidad **9**

11. Rendimiento y operación **9**

12. Publicación y distribución **10**

12.1. Ejecución local . . . 10

12.2. Enlace temporal . . . 10

12.3. Hugging Face Spaces 10
12.4. Infraestructura propia 10

13. Criterios de selección **10**

14. Conclusiones **11**

A. Estructura entregada **11**

B. Comandos de instalación y ejecución **11**

1. Resumen ejecutivo

Gradio es un paquete de código abierto orientado a convertir funciones, modelos y servicios de Python en aplicaciones web interactivas. Su principal ventaja consiste en reducir la distancia entre una pieza de lógica computacional y una experiencia que puede ser utilizada por una persona no especializada. El artículo original presentó esta idea como una respuesta al problema de accesibilidad y retroalimentación en proyectos de aprendizaje automático [1].

Este informe estudia Gradio como herramienta principal para dashboards interactivos, lo compara con Streamlit, Dash, Panel y Shiny, explica su modelo de programación y examina aspectos que suelen omitirse en un ejemplo básico: estado, eventos, colas, concurrencia, publicación, seguridad de archivos y criterios de uso en producción. Como aplicación práctica, se desarrolló un observatorio comercial con 576 registros sintéticos, filtros coordinados, cinco indicadores, cuatro visualizaciones, hallazgos automáticos y exportación CSV.

Resultado central

Gradio resulta especialmente conveniente cuando el valor principal ya existe como función, modelo o pipeline de Python y se necesita una interfaz demostrable con rapidez. No sustituye automáticamente un sistema de inteligencia de negocios ni un frontend empresarial hecho a medida; su idoneidad depende de los requisitos de gobierno, diseño, escala y operación.

2. Objetivos y alcance

2.1. Objetivo general

Investigar y demostrar el uso de Gradio para construir dashboards interactivos, identificando su arquitectura, ecosistema, fortalezas, limitaciones y posibilidades de despliegue.

2.2. Objetivos específicos

- Definir qué distingue a un dashboard interactivo de un reporte estático.
- Explicar el papel de `Interface`, `Blocks`, componentes y eventos.
- Comparar Gradio con alternativas relevantes del ecosistema de datos.
- Construir una aplicación más cercana a un caso analítico real que a un simple “Hola mundo”.
- Documentar instalación, pruebas, publicación, rendimiento y seguridad.
- Producir artefactos reproducibles: código Python, presentación HTML e informe PDF.

El proyecto utiliza datos sintéticos; no pretende realizar inferencias sobre una empresa real. Las conclusiones técnicas se apoyan en documentación oficial consultada en octubre de 2026 y en pruebas locales con Gradio 6.30.0.

3. Dashboards interactivos

Un dashboard interactivo reúne indicadores, tablas y visualizaciones en una interfaz que responde a acciones del usuario. La interacción permite formular preguntas sucesivas: seleccionar un periodo, aislar una región, comparar categorías, cambiar la métrica o descargar el subconjunto resultante. Por

ello, el dashboard no es solamente una colección de gráficas; es un mecanismo de exploración y comunicación.



Una solución útil debe equilibrar cinco dimensiones: claridad visual, capacidad analítica, tiempo de desarrollo, facilidad de publicación y sostenibilidad operativa. Una herramienta excelente para prototipos puede no ser la mejor elección para un portal corporativo con cientos de usuarios, identidad visual estricta y controles avanzados de acceso.

4. Qué es Gradio

Gradio es un paquete abierto de Python que genera interfaces web para funciones, modelos de aprendizaje automático, APIs y flujos arbitrarios. El inicio rápido oficial enfatiza que una aplicación básica no requiere conocimientos de JavaScript, CSS ni alojamiento web para comenzar [2]. El proyecto se distribuye bajo licencia Apache 2.0 y su desarrollo ocurre públicamente en GitHub [10].

4.1. Origen y propósito

Abid et al. propusieron Gradio en 2019 para facilitar que especialistas de dominio pudieran probar modelos y aportar retroalimentación sin reproducir el entorno técnico del investigador [1]. Ese propósito explica varias decisiones de diseño actuales:

- declarar entradas y salidas de forma explícita;
- soportar texto, imágenes, audio, video, archivos, tablas y gráficas;
- compartir una aplicación mediante una URL;
- ejecutar la lógica en Python y presentar el resultado en el navegador;
- ofrecer clientes programáticos además de la interfaz visual.

4.2. Lenguajes y ecosistema

Python es el lenguaje principal de autoría. El paquete `gradio_client` permite consumir aplicaciones desde Python, mientras que `@gradio/client` habilita consumo desde JavaScript o TypeScript. Una aplicación alojada puede actuar simultáneamente como interfaz humana y como endpoint programático. Hugging Face Spaces ofrece integración directa para alojar demos Gradio en repositorios versionados [11].

5. Modelo de programación

5.1. Tres niveles principales

Abstracción	Propósito	Uso apropiado
`Interface`	Envuelve una función con componentes de entrada y salida.	Demos pequeñas, prototipos y validación rápida.
`Blocks`	Controla layout, eventos y flujos entre múltiples componentes.	Dashboards, herramientas con pestañas y aplicaciones coordinadas.
`ChatInterface`	Implementa el patrón conversacional con historial y streaming.	Asistentes, chatbots y experiencias multimodales.

La documentación describe `Blocks` como la API de menor nivel que ofrece mayor control sobre layout, eventos y flujos de datos [3]. Los componentes funcionan como nodos visibles; los eventos conectan esos nodos con funciones de Python.

5.2. Ejemplo mínimo

Interfaz mínima con Gradio

```python
import gradio as gr

def saludar(nombre, intensidad):
    return f"Hola, {nombre}" + "!" * int(intensidad)

demo = gr.Interface(
    fn=saludar,
    inputs=[gr.Textbox(label="Nombre"), gr.Slider(1, 5, step=1)],
    outputs=gr.Textbox(label="Resultado"),
)

demo.launch()
```

El patrón consta de cuatro pasos: escribir una función, declarar componentes, conectarlos y lanzar el servidor. En desarrollo, `gradio app.py` habilita recarga automática; `python app.py` inicia la aplicación normalmente [2].

6. Arquitectura y flujo de datos

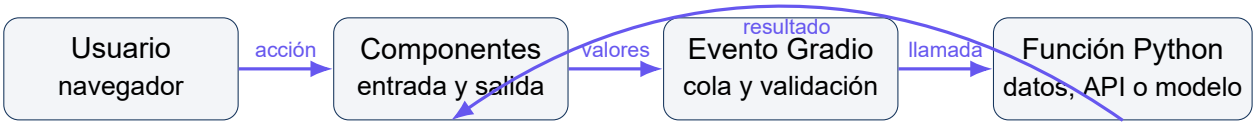


Figura 1: Flujo simplificado de una interacción en Gradio.

La función recibe valores ya serializados por los componentes y devuelve resultados que Gradio transforma para el navegador. Este contrato facilita reemplazar una función simple por una operación más compleja sin rediseñar toda la interfaz.

6.1. Estado

La documentación distingue tres alcances [5]:

- **Global**: compartido por todos los usuarios mientras el proceso está activo.
- **Sesión**: aislado por usuario durante la sesión mediante `gr.State`.
- **Navegador**: persistente en almacenamiento local mediante `gr.BrowserState`.

Elegir el alcance incorrecto puede mezclar información entre usuarios o perder preferencias al recargar. El estado global es conveniente para recursos inmutables, como un modelo cargado una vez, pero no para datos privados de una sesión.

6.2. Colas y concurrencia

Cada listener dispone de una cola. El parámetro `concurrency_limit` controla cuántas ejecuciones simultáneas admite un evento; los listeners también pueden compartir una cola mediante un identificador común [4]. Esto es relevante cuando una operación consume GPU, memoria significativa o una API limitada. Aumentar concurrencia sin medir recursos puede elevar la latencia o producir errores por agotamiento de memoria.

7. Comparación con otras herramientas

Herramienta	Lenguaje	Fortalezas	Consideraciones
Gradio	Python; clientes Python y JS/TS	Interfaces rápidas para modelos, funciones, multimedia y APIs; publicación sencilla; API automática.	Menor control que un frontend a medida; debe evaluarse seguridad, concurrencia y operación.
Streamlit	Python	Modelo de ejecución accesible, ecosistema amplio para aplicaciones de datos y prototipos.	El patrón de reejecución requiere comprender estado y caché en aplicaciones grandes [12].
Dash	Python	Control explícito mediante callbacks, integración natural con Plotly y aplicaciones analíticas.	Puede requerir más estructura y código para prototipos pequeños [13].
Panel	Python	Integración profunda con PyData, HoloViz, notebooks y múltiples librerías de visualización.	Flexibilidad amplia implica más decisiones arquitectónicas [14].
Shiny	R y Python	Modelo reactivo maduro, modularidad y fuerte tradición estadística.	La elección suele depender del ecosistema R/Python y de la plataforma de publicación [15].

Criterio práctico

Si el núcleo es un modelo o una función Python que debe demostrarse pronto, Gradio suele reducir el tiempo hasta la primera versión. Si el núcleo es un producto web con navegación compleja, diseño altamente personalizado o gobierno empresarial, conviene comparar una arquitectura web completa o herramientas analíticas más especializadas.

8. Aplicación desarrollada por el grupo

El proyecto implementa un *observatorio comercial* con datos sintéticos de 2026. El dataset se genera de manera determinista para asegurar que la exposición y las pruebas produzcan siempre los mismos resultados.

576	12	4	4 x 3
observaciones	meses	regiones	categorías y canales

8.1. Variables y funcionalidades

Cada observación contiene fecha, región, categoría, canal, ventas, costo, margen bruto, unidades, pedidos y satisfacción. La interfaz incorpora:

- filtros multiselección por región, categoría y canal;
- mes inicial, mes final y meta mensual configurable;
- selección de métrica y tipo de visualización;
- cinco KPIs: ventas, crecimiento, margen, ticket y cumplimiento;
- tendencia mensual, mezcla por categoría, rendimiento regional y mapa de calor;
- hallazgos narrativos generados desde el subconjunto filtrado;
- tabla detallada y descarga CSV;
- restablecimiento de estado y cola con límite de concurrencia.



Figura 2: Vista validada del observatorio comercial construido con Gradio 6.30.0.

8.2. Cálculos principales

El margen bruto porcentual se calcula como

$$\text{Margen bruto \%} = \frac{\sum(\text{Ventas} - \text{Costo})}{\sum \text{Ventas}} \times 100.$$

El ticket promedio se define como ventas totales divididas entre pedidos. El crecimiento compara el primer y último mes visible. Para que la meta siga siendo interpretable al filtrar categorías o canales, se ajusta proporcionalmente a la cobertura seleccionada.

8.3. Calidad y pruebas

Se implementaron cuatro pruebas automáticas: cobertura completa del dataset, exactitud del filtrado, contrato de ocho salidas coordinadas y creación de un CSV filtrado. Adicionalmente, se verificó la aplicación en un navegador real a 1920 x 1080, se comprobó el cambio de métrica y la navegación por pestañas, y se confirmó la ausencia de errores en consola.

9. Diseño visual y accesibilidad

Gradio ofrece temas predefinidos y un motor de variables visuales; los temas pueden extenderse y el CSS puede suministrarse al lanzar la aplicación [6]. La documentación advierte que selectores acoplados al DOM interno pueden cambiar entre versiones, por lo que conviene utilizar clases propias y selectores estables [7].

En la aplicación se adoptaron las siguientes decisiones:

- una paleta oscura consistente, independiente de la preferencia del navegador;
- contraste alto para texto, estados seleccionados y controles;
- grids responsivos con `minmax(0, 1fr)` para eliminar desbordamientos;
- etiquetas descriptivas y orden lógico de navegación;
- color acompañado de texto, no utilizado como único portador de significado;
- visualizaciones con títulos, ejes, leyendas y *tooltips* claros.

10. Seguridad y privacidad

Compartir una aplicación vuelve accesible una superficie que procesa entradas externas. La guía oficial explica qué archivos pueden entrar en el caché y recomienda restringir `allowed_paths`, utilizar `blocked_paths` y limitar el tamaño de archivos subidos [8].

Lista mínima antes de publicar

- No incluir claves, contraseñas o tokens en el repositorio.
- Validar entradas y no convertir texto del usuario directamente en una ruta local.
- Mantener `allowed_paths` lo más reducido posible.
- Establecer `max_file_size` cuando existan cargas de archivos.
- Añadir autenticación si los datos o resultados no son públicos.
- Ejecutar con una cuenta de sistema con permisos mínimos.
- Registrar errores sin exponer trazas o secretos al usuario final.

Los enlaces temporales creados con `share=True` son públicos; no deben considerarse un control de acceso. La aplicación continúa ejecutándose en la computadora local y el enlace actúa como túnel mientras el proceso está activo [9].

11. Rendimiento y operación

El rendimiento depende de la operación conectada a cada evento. Filtrar cientos de filas requiere pocos recursos; ejecutar un modelo generativo grande puede necesitar GPU y serialización cuidadosa. La documentación de colas propone ajustar concurrencia por evento y compartir colas cuando varias funciones compiten por el mismo recurso [4].

Buenas prácticas operativas:

1. cargar modelos y datos estáticos una sola vez, fuera del callback;
2. aplicar caché a resultados deterministas costosos;
3. limitar concurrencia según mediciones, no por intuición;
4. paginar o resumir tablas grandes antes de enviarlas al navegador;
5. fijar versiones en `requirements.txt`;

6. monitorear latencia, errores, memoria y longitud de cola;
7. separar desarrollo, pruebas y producción.

12. Publicación y distribución

12.1. Ejecución local

La opción más segura para una exposición presencial es ejecutar localmente:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python app.py
```

12.2. Enlace temporal

`python app.py --share` crea una URL pública temporal. Es útil para revisión rápida, pero depende del equipo local y no sustituye un despliegue persistente [9].

12.3. Hugging Face Spaces

Spaces almacena el proyecto como repositorio y reconstruye la aplicación con cada actualización. Admite visibilidad pública, protegida o privada, sujeta al plan y configuración correspondiente [11]. Una app Gradio alojada también puede exponerse como endpoint para clientes programáticos.

12.4. Infraestructura propia

Cuando existen requisitos específicos de red, observabilidad o autenticación, Gradio puede ejecutarse en infraestructura propia y montarse junto a FastAPI. En ese escenario deben definirse proxy inverso, TLS, límites de recursos, escalado, almacenamiento temporal y estrategia de actualización.

13. Criterios de selección

Escenario	Gradio	Framework analítico	Frontend a medida
Demo de modelo de IA	**Muy adecuado**	Adecuado	Costoso para iniciar
Prototipo de función Python	**Muy adecuado**	Adecuado	Costoso para iniciar
Dashboard interno mediano	Adecuado	**Muy adecuado**	Adecuado
Portal empresarial complejo	Posible con evaluación	Adecuado	**Muy adecuado**
Identidad visual estricta	Limitado a moderado	Moderado	**Muy adecuado**
Entrega en pocas horas	**Muy adecuado**	Adecuado	Poco adecuado

La tabla no representa una regla absoluta. La experiencia del equipo, el alojamiento disponible y el tipo de usuario pueden cambiar la decisión. La recomendación es construir un prototipo pequeño y medir requisitos reales antes de comprometer una arquitectura de producción.

14. Conclusiones

Gradio reduce de forma notable el trabajo necesario para convertir lógica de Python en una aplicación interactiva. Su propuesta sigue alineada con el problema descrito en el trabajo original: permitir que colaboradores y usuarios de dominio prueben sistemas computacionales sin instalar el entorno del desarrollador.

La investigación muestra que su simplicidad inicial no elimina responsabilidades de ingeniería. Una aplicación pública requiere decisiones sobre estado, cola, autenticación, archivos, recursos y observabilidad. Asimismo, un buen dashboard necesita jerarquía visual, indicadores interpretables y controles que respondan a preguntas reales.

La aplicación del grupo demuestra un punto intermedio útil: conserva la rapidez de Gradio, pero incorpora segmentación multidimensional, cálculos coordinados, narrativa automática, exportación y pruebas. Por tanto, Gradio se recomienda para demos de IA, prototipos de datos, docencia y herramientas internas; para productos empresariales complejos debe compararse con frameworks analíticos o una solución web a medida.

A. Estructura entregada

Archivo	Propósito
`app.py`	Aplicación Gradio y generación del dataset sintético.
`requirements.txt`	Versiones y dependencias del proyecto.
`tests/test_app.py`	Pruebas de datos, filtros, salidas y exportación.
`presentacion_gradio.html`	Presentación interactiva autocontenida.
`INVESTIGACION.md`	Síntesis en Markdown y guion de exposición.
`informe_gradio.tex`	Fuente LaTeX de este documento.
`informe_gradio.pdf`	Informe compilado para entrega.

B. Comandos de instalación y ejecución

```
cd "C:\Users\Daniel Barillas\Desktop\Gradio_Data"
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Para ejecutar las pruebas:

```
python -m unittest discover -s tests -v
```

Referencias

[1] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan y J. Zou, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild”, arXiv:1906.02569, 2019. https://arxiv.org/abs/1906.02569

[2] Gradio, “Quickstart”, documentación oficial. https://www.gradio.app/guides/quickstart

[3] Gradio, “Blocks”, referencia oficial. https://www.gradio.app/docs/gradio/blocks

[4] Gradio, “Queuing”, guía oficial. https://www.gradio.app/guides/queuing

[5] Gradio, “State in Blocks”, guía oficial. https://www.gradio.app/main/guides/state-in-blocks

[6] Gradio, “Theming Guide”, documentación oficial. https://www.gradio.app/guides/theming-guide

[7] Gradio, “Custom CSS and JS”, documentación oficial. https://www.gradio.app/guides/custom-CSS-and-JS

[8] Gradio, “Security and File Access”, documentación oficial. https://www.gradio.app/guides/file-access

[9] Gradio, “Sharing Your App”, documentación oficial. https://www.gradio.app/guides/sharing-your-app

[10] Gradio, repositorio oficial. https://github.com/gradio-app/gradio

[11] Hugging Face, “Spaces Overview” y “Gradio Spaces”. https://huggingface.co/docs/hub/spaces-overview

[12] Streamlit, documentación oficial. https://docs.streamlit.io/

[13] Plotly, “Dash Documentation”. https://dash.plotly.com/

[14] HoloViz, “Panel Documentation”. https://panel.holoviz.org/

[15] Posit, “Shiny for Python”. https://shiny.posit.co/py/